CSE475 Machine Learning

Task 1 Report: Understanding the Data

Graph-Based Network Intrusion Detection on UNSW-NB15

Group 7 · Track 1: Graph Neural Network

Member	Student ID
Mohammed Afnan Ahmad Bhuiyan	2024-1-60-228
Md. Jakaria Masud	2023-3-60-386
Tawhid Hossain Tomal	2023-1-60-197
Naima Islam	2022-3-60-215

Department of Computer Science and Engineering, East West University

Fall 2026

Section:01

Code: github.com/afnan-46/Group7_UNSWNB15_CSE475

Summary

We analysed 700,000 network flows from the full UNSW-NB15 release to prepare a graph neural network that classifies traffic flows, with hosts as nodes and flows as edges. Five findings shape the rest of the project.

- **Severe imbalance.** 85.73% of flows are normal and Worms has only 53 flows, an 11,322 : 1 ratio. A model that always predicts Normal scores 85.73% accuracy but about 9% macro-F1, so we evaluate on macro-F1.
- **A real graph exists.** The data contains 48 hosts connected by 292 distinct host pairs, carrying all 700,000 flows. The structure is in the data, not invented.
- **IP addresses leak the label.** All attack traffic comes from four IP addresses. A classifier given only the source and destination IPs reaches 0.965 macro-F1 while learning nothing about attacks, so IPs define the graph but are never used as features.
- **Data quality needs care.** 20.4% of rows are exact duplicates, two features are about half missing, and 11 feature pairs are almost perfectly correlated.
- **A host-disjoint split is feasible.** Splitting by host gives 72 / 13 / 15% with zero host overlap and every class in every part, at the cost of a large shift in class mix between parts.

The literature review finds that no published graph model on UNSW-NB15 is evaluated on hosts unseen in training, two rebalance the data before testing, and the only controlled test of whether graph structure helps used a synthetic topology. Our research question follows from that gap (Section 7).

1. Introduction

A conventional intrusion detector treats each network flow as an isolated row of numbers. That discards information that is clearly relevant: a port scan is one host contacting many others, and a denial-of-service attack is many hosts converging on one target. That pattern lives in the connections between flows, not inside any single flow.

Our project represents network traffic as a graph, in which each host is a node and each flow is an edge carrying that flow's measurements, and trains a graph neural network to classify each edge. When classifying a flow, the model can draw on both the flow's own features and the behaviour of the two hosts it connects. This report covers Task 1: what the data looks like, what that implies for preprocessing and evaluation, what published work has done, and the gap our experiment addresses.

2. Dataset

UNSW-NB15 was created at the Australian Centre for Cyber Security, UNSW Canberra (Moustafa & Slay, 2015). The IXIA PerfectStorm tool generated a mix of real normal traffic and synthetic attacks; around 100 GB of raw packets were captured, and flow records were extracted with Argus, Bro-IDS and twelve additional algorithms. It was designed to replace KDD'99 and NSL-KDD, whose 1998 traffic no longer reflects modern attacks.

Two releases exist. The partitioned release (175,341 training and 82,332 testing rows) has 45 features but no IP or port columns. The full release (2,540,044 rows in four files) has all 49 features, including source and destination IPs. Because our graph uses hosts as nodes, we use the full release.

Property	Value
Source	ACCS, UNSW Canberra; Kaggle mirror mrwellsdavid/unsw-nb15
Release used	Full release, files UNSW-NB15_1.csv to UNSW-NB15_4.csv
Rows analysed	700,000 (first 175,000 rows of each of the four files)
Columns	49
Modellable features	37 numeric, 4 categorical (proto, state, service, ct_ftp_cmd)
Identifier columns	6: srcip, sport, dstip, dsport, stime, ltime
Targets	label (binary) and attack_cat (10 classes)
Distinct hosts	48 (42 source, 46 destination)
Missing values	675,596 cells, in 2 columns
Duplicate rows	142,474 (20.35%)

We analysed 700,000 of the 2,540,044 rows to keep memory within Kaggle's limits. The four files are in chronological order, so taking the first 175,000 rows of each gives four time-contiguous windows rather than a random sample. The attack share in our subset (14.27%) is close to that of the full release (12.65%). Task 2 will use the full release where memory allows.

3. Exploratory data analysis

Each finding is followed by what it changes about preprocessing or evaluation. All results come from *Group7_UNSWNB15_task1_eda.ipynb*.

3.1 Distributions and outliers

Most flow features are strongly right-skewed and heavy-tailed. Of the 37 numeric features, 29 have |skewness| above 2 and 22 have kurtosis above 10. The most extreme is trans_depth (skewness 212.2): almost every flow has a value of 0, and a handful have values up to 172. Byte counts behave the same way. Median sbytes is 1,036, but the maximum is 13.7 million. By the 1.5 × IQR rule, 30 of 37 features have more than 5% outliers.

Implication: these outliers are genuine traffic, not errors, so we will transform rather than remove them. Features with |skewness| above 2 receive a log1p transform, and all features are then standardised. The scaler is fitted on the training split only.

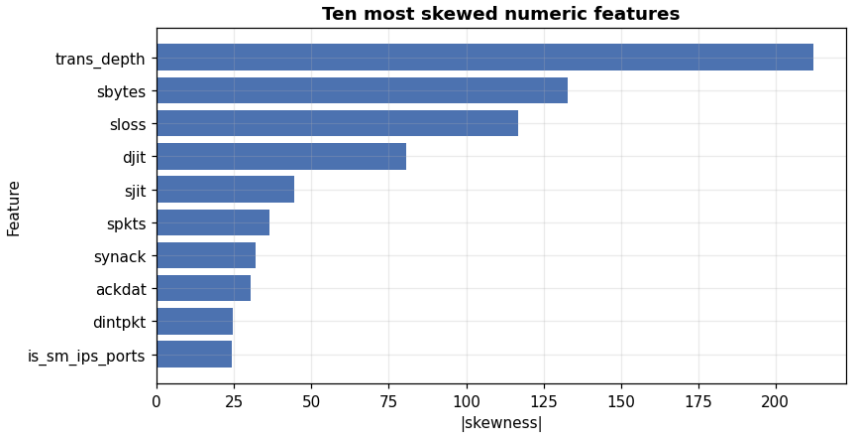


Figure 1. The ten most skewed numeric features.

3.2 Data quality

Check	Result	Action
Missing values	is_ftp_login 49.30%; ct_flw_http_mthd 47.21%	Impute with training-split statistics plus a missing-indicator flag
Text in a numeric column	ct_ftp_cmd read as text	Coerce to numeric before modelling
Exact duplicates	142,474 rows (20.35%)	De-duplicate before splitting
Duplicates ignoring identifiers	150,284 rows (21.47%)	Same; these are identical to the model
Near-constant features	is_sm_ips_ports (99.83% zero)	Keep for now; check per-class value
Pairs with |r| > 0.95	11 pairs	Drop one of each pair before SHAP (Task 3)

Missing values are concentrated in two fields that describe FTP logins and HTTP methods. That pattern is consistent with the fields being undefined for flows of other protocols, rather than lost at random. A missing-indicator flag preserves that information instead of hiding it behind an imputed number.

The duplicate rate is the most consequential finding in this section. One flow in five has an exact copy. If the data is split first, copies of the same row will land on both sides of the split, and the model will be tested on rows it has already seen.

Implication: we de-duplicate before splitting, and fit every imputer, encoder and scaler on the training split only, refitting inside each cross-validation fold.

3.3 Class balance

Class	Flows	Share	Ratio to smallest
Normal	600,079	85.726%	11,322 : 1
Generic	67,565	9.652%	1,275 : 1
Exploits	13,052	1.865%	246 : 1
Fuzzers	8,824	1.261%	167 : 1
Reconnaissance	4,189	0.598%	79 : 1
DoS	3,899	0.557%	74 : 1
Analysis	945	0.135%	18 : 1
Backdoor	907	0.130%	17 : 1
Shellcode	487	0.070%	9 : 1
Worms	53	0.008%	1 : 1

Attacks make up 14.27% of flows (99,921 of 700,000). Among them, Generic and Exploits dominate and five classes have fewer than 1,000 examples each. A model that always predicts Normal would score 85.73% accuracy but only about 9.2% macro-F1.

Implication: accuracy is meaningless here, so we judge every model on macro-F1 and per-class recall, as the brief requires. We will use a class-weighted loss and expect low recall on Worms and Shellcode regardless, which we will report rather than hide.

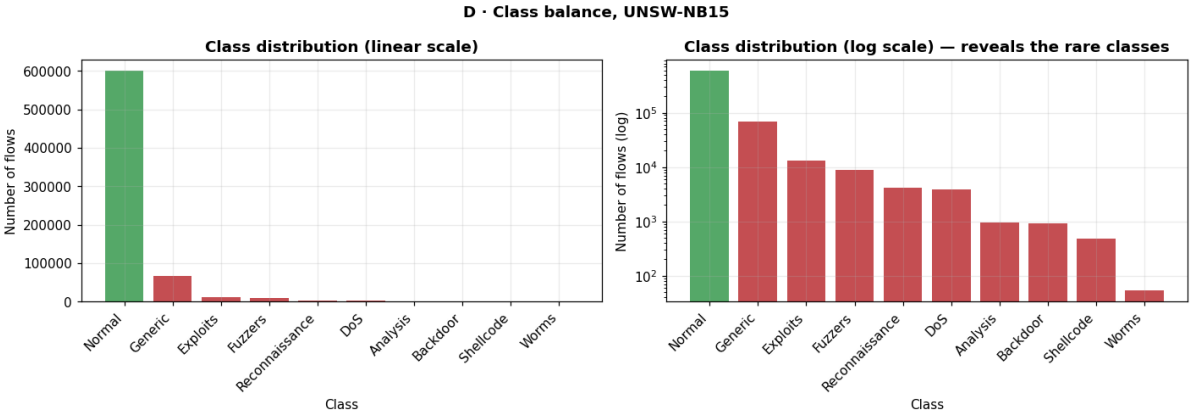


Figure 2. Class distribution on linear and log scales. On the linear scale the rare classes are invisible.

3.4 Which features separate the classes

Ranked by ANOVA F-statistic against the attack category, the strongest features are sttl (source time-to-live), ct_state_ttl and dttl, followed by the ct_* connection counters. In Figure 3, normal and attack traffic separate cleanly on some of these features and overlap heavily on others.

The dominance of TTL features deserves caution. TTL is set by the sending host's operating system and network distance. In a testbed where attacks come from a few fixed machines, a TTL value may partly identify the attacking machine rather than the attack. We will test this in Task 2 by checking how much performance depends on the TTL features.

Implication: a flat classifier on these features will already be strong, so our baselines will be hard to beat. The graph model has to add value on the classes that flow features do not separate.

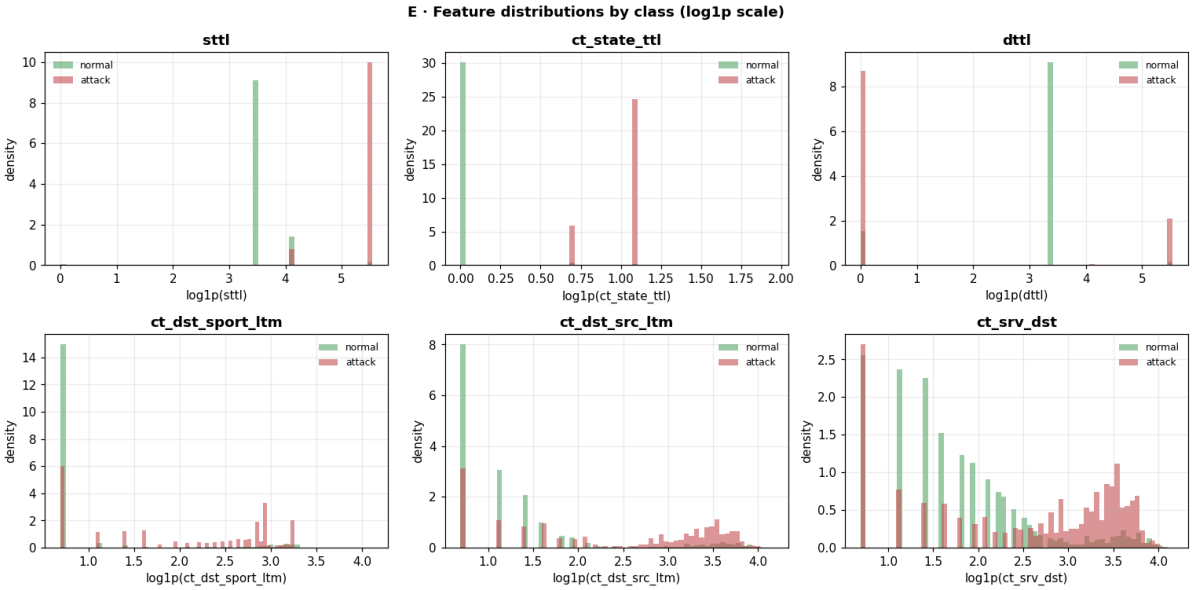


Figure 3. Distributions of the six most discriminative features, normal versus attack (log1p scale).

3.5 Correlation and redundancy

Eleven feature pairs have |r| above 0.95. The strongest are swin and dwin (0.996), dbytes and dloss (0.992), and dloss and dpkts (0.992). The ct_* connection counters form a tightly correlated block, because several count overlapping windows of recent connections. Eight features are candidates for removal: ct_dst_src_ltm, ct_src_dport_ltm, ct_src_ltm, ct_srv_dst, dloss, dpkts, dwin and sloss.

Implication: correlated features split credit between themselves in SHAP explanations, making them unstable. We will drop one feature from each highly correlated pair before the explainability analysis in Task 3.

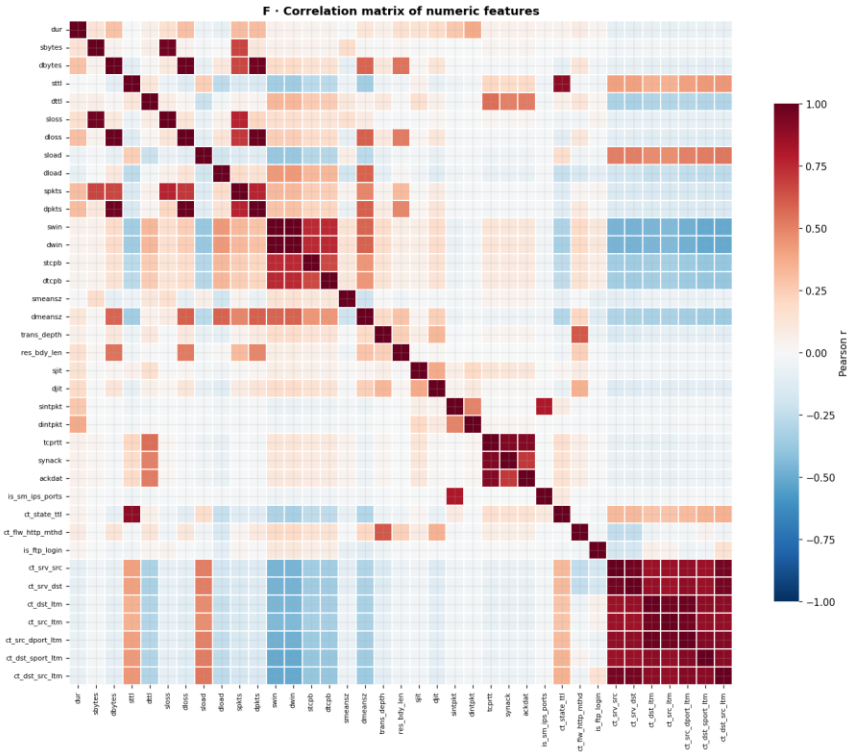


Figure 4. Correlation matrix of the 37 numeric features.

3.6 Low-dimensional structure

Fourteen principal components explain 90% of the variance, so the 37 features carry far fewer independent dimensions than their count suggests. In the t-SNE projection (Figure 5), normal flows form compact clusters well separated from attacks, and Generic forms distinct clusters of its own. Exploits, DoS, Analysis, Backdoor and Reconnaissance, however, are intermixed across many small clusters.

Implication: separating normal from attack is relatively easy; separating attack families from one another is not. The intermixed families are where flow features fall short, and they are where we expect information about the hosts involved to help most.

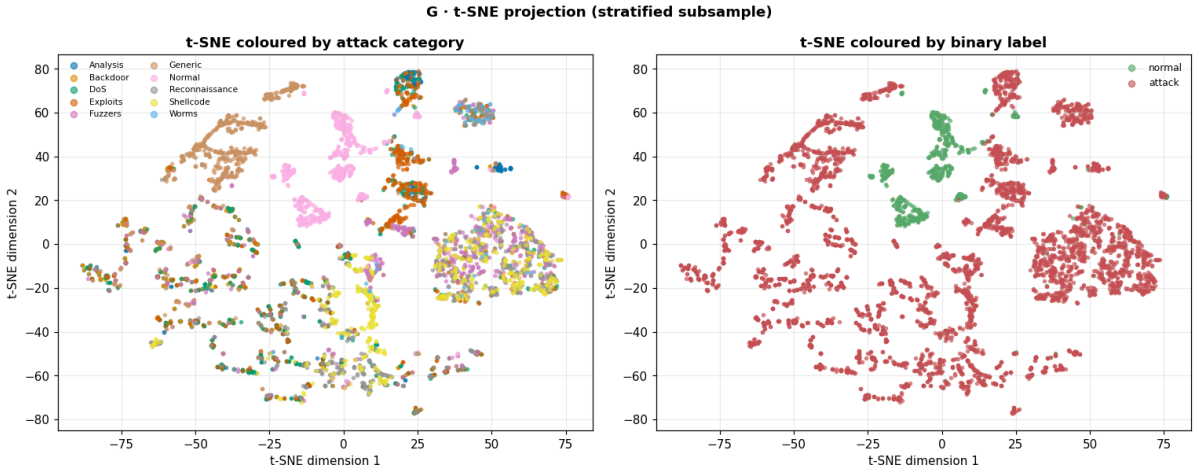


Figure 5. t-SNE projection of a stratified subsample (5,340 flows), coloured by attack category (left) and binary label (right).

4. Findings for a graph-based approach

The following analyses go beyond the standard checklist. They test whether a host graph exists in this data, whether it can be used honestly, and whether a fair evaluation is possible.

4.1 The data contains a real graph

Graph property	Value
Hosts (nodes)	48
Source hosts / destination hosts	42 / 46
Distinct host pairs	292
Flows (edges)	700,000
Mean flows per host pair	2,397
Graph density	0.129
Source hosts carrying 80% of flows	12 of 42 (28.6%)

The host graph is small in nodes but very rich in edges: 48 hosts exchange 700,000 flows over 292 host pairs. That makes it a directed multigraph, where each host pair carries many flows. Traffic is concentrated, with 12 source hosts producing 80% of all flows.

Implication: the graph comes from the data, which is what the brief requires for a graph to be justified. Its small node count matters, though. A host-disjoint test set will contain few hosts, so we will report results across several host splits rather than trust a single one.

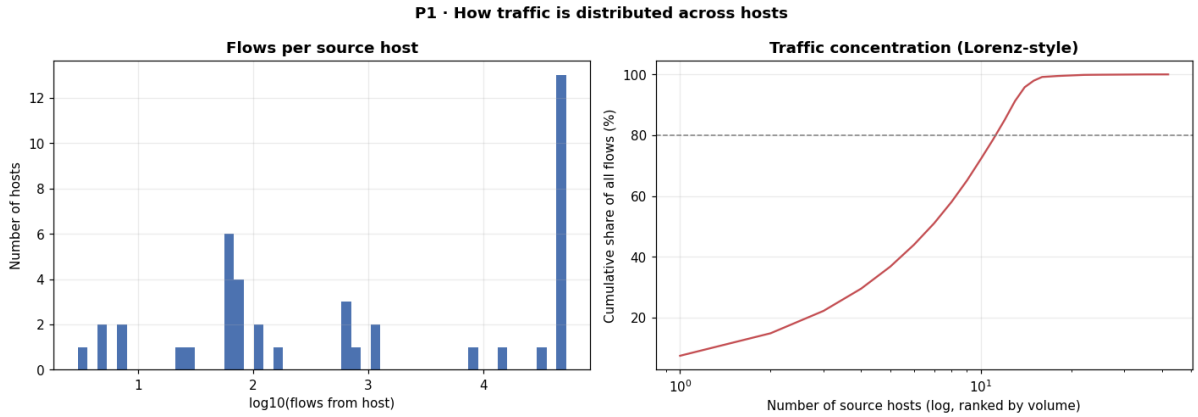


Figure 6. Flows per source host (left) and cumulative share of flows by host (right).

4.2 IP addresses leak the label

Every attack flow in our sample comes from one of four IP addresses: 175.45.176.0 to 175.45.176.3. These four hosts also send some normal traffic, between 70% and 93% of their flows being attacks. The other 38 source hosts send normal traffic only.

To measure what this means, we trained a decision tree using only the source and destination IP addresses as input. It reached a macro-F1 of 0.965 (± 0.001 over three folds) without seeing any feature that describes network behaviour.

Implication: any model given IP addresses as features will appear to work while learning to recognise four machines, not attacks. We therefore exclude srcip, dstip, sport, dsport, stime and ltime from every model's input. IP addresses are used only to decide which flows share a node in the graph, which carries no label information by itself.

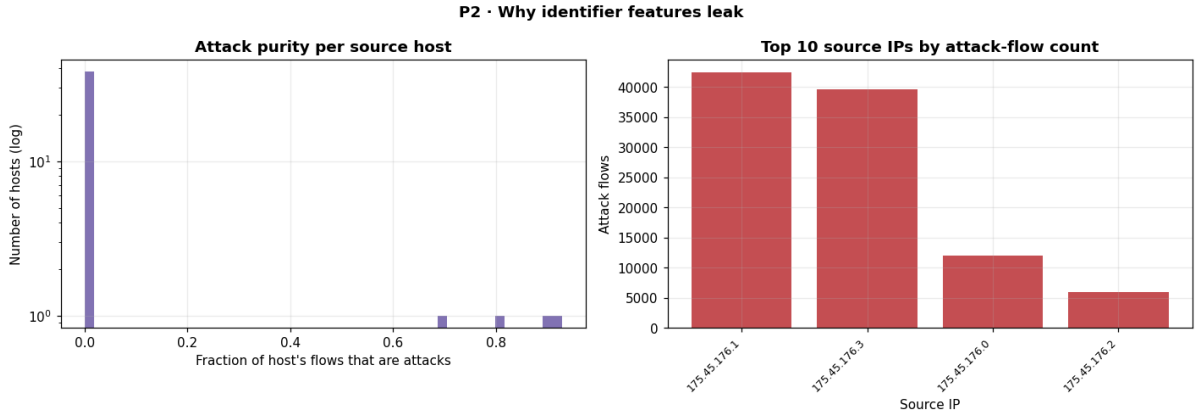


Figure 7. Share of attack traffic per source host (left) and attack flows per attacking IP (right).

4.3 A host-disjoint split is feasible but shifts the class mix

The course requires separate training, validation and test sets, with the test set untouched until the final evaluation. We split by source host so that no host appears in more than one part.

Part	Flows	Share	Source hosts
Training	505,219	72.17%	29
Validation	89,839	12.83%	6
Test	104,942	14.99%	7

Host overlap between parts is zero, and every one of the ten classes appears in all three parts, including Worms (7 / 13 / 33 flows). But the class mix shifts sharply. Normal is 91.6% of the training set and only 50.9% of the test set, while Generic rises from 6.6% to 32.5%. This happens because each attacking host sends a characteristic mix of attacks, and the four attacking hosts are spread unevenly across the three parts.

We also compared the two kinds of split directly. A Random Forest on 60,000 flows scored 0.981 macro-F1 under a random split and 0.972 under a host-disjoint split, a gap of 0.009. For binary classification with flow features, the optimism from random splitting is therefore small in this data.

Implication: we will use the three-way host-disjoint split throughout. The small binary gap suggests the larger risks to a fair result are identifier features (Section 4.2) and rebalancing, not splitting alone. Whether the gap stays small for multiclass and minority classes is an open question we will test in Task 2.

5. Related work

We reviewed five papers from 2022 to 2026, each verified against its full text. The complete review is in *related_work/Group7_UNSWNB15_related_work_table.pdf*. The table below summarises them by how each was evaluated, because evaluation conditions decide whether a reported score can be compared with ours.

Paper	Model	Evaluation conditions	Reported result
E-GraphSAGE (Lo et al., 2022)	Edge-featured GraphSAGE	Full data; random 70/30; no validation set	Multiclass 98.15% acc., weighted F1 0.98
FN-GNN (Tran & Park, 2024)	GCN + SAGEConv	50k subset rebalanced to 61/39; random 80/20	98.65% acc., weighted F1 98.65%
DIGNN-A (Liu & Guo, 2025)	Dynamic GNN on line graphs	Full data; 0.3 test ratio; no validation set	Binary 99.3% acc., F1 97.22%
GCN-KNN (Ansah et al., 2025)	GCN fused with k-NN	44k subset rebalanced to 50/50; split not stated	99.94% acc., F1 99.93%

Paper	Model	Evaluation conditions	Reported result
Nwachukwu (2026)	Graph features + classical ML	Synthetic topology; split not stated	Graph features lowered RF F1 from 0.940 to 0.931

We excluded Anomal-E (Caville et al., 2022) from the comparison after confirming it uses NF-UNSW-NB15-v2, a 43-feature NetFlow re-extraction of the same traffic. Its scores are not comparable with results on the original release.

6. Research gap

The review shows three weaknesses in how graph models have been evaluated on this dataset.

- **No paper evaluates on unseen hosts.** E-GraphSAGE and DIGNN-A use 70/30 splits without a validation set, FN-GNN uses a random 80/20 split, and the other two do not state their split. Section 4.2 shows why this matters here: all attacks come from four machines, and a model that can see those machines in training can learn to recognise them.
- **Two papers rebalance the data.** FN-GNN rebalances to 61/39 and GCN-KNN to 50/50, while the real data is about 86/14 (Section 3.3). Their 98.65% and 99.94% are measured on an easier problem than the one the dataset poses.
- **The one controlled test used a synthetic topology.** Nwachukwu (2026) is the only study that isolates whether graph information helps, and finds it does not. But its graph was built from generated IP addresses, not real ones, and no graph neural network was tested. The author notes that real topologies may give different results.

It is therefore unknown whether the real host topology in UNSW-NB15 improves detection when a graph neural network is evaluated on hosts unseen in training, on the natural class distribution, and with identifier features excluded. Because the comparison papers report weighted F1 or binary metrics, we will report macro-F1 alongside those metrics and compare like with like.

7. Research question

> **Does a host-topology graph improve macro-F1 over a k-nearest-neighbour feature-similarity graph on UNSW-NB15, under the same three-way host-disjoint split and the same training budget?**

The question changes one factor only: the rule that decides which flows are connected. In the host graph, flows are connected when they share a real host. In the k-NN graph, flows are connected when their features are similar. Everything else is held fixed: the features, the split, the model and the training budget.

Either answer is informative. If the host graph wins, real network structure carries information beyond what flow features capture. If it does not, the flow features already encode most of what the structure would add, which is what Nwachukwu (2026) found with hand-computed graph features.

8. Plan for Task 2

- De-duplicate, coerce ct_ftp_cmd to numeric, impute the two missing fields with indicator flags, apply log1p to skewed features, and standardise, all fitted on the training split only.
- Train Logistic Regression, SVM, Random Forest, MLP and XGBoost baselines on the three-way host-disjoint split, reporting accuracy, precision, recall, macro- and weighted F1, confusion matrices, ROC and precision–recall curves, and training time.

- Build the host graph from training flows only, add validation and test flows as new edges at inference time, and train an E-GraphSAGE-style edge classifier in PyTorch Geometric, choosing checkpoints on validation macro-F1.
- Check how much performance depends on the TTL features, and report results across several host splits because of the small number of hosts.

We expect XGBoost to be difficult to beat, and state this before running the experiment.

References

[1] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems," Military Communications and Information Systems Conference (MilCIS), 2015.

[2] W. W. Lo, S. Layeghy, M. Sarhan, M. Gallagher and M. Portmann, "E-GraphSAGE: A graph neural network based intrusion detection system for IoT," IEEE/IFIP NOMS, 2022. arXiv:2103.16329.

[3] D.-H. Tran and M. Park, "FN-GNN: A novel graph embedding approach for enhancing graph neural networks in network intrusion detection systems," Applied Sciences, 14(16), 6932, 2024. doi:10.3390/app14166932.

[4] J. Liu and M. Guo, "DIGNN-A: Real-time network intrusion detection with integrated neural networks based on dynamic graph," Computers, Materials & Continua, 82(1), 817–842, 2025. doi:10.32604/cmc.2024.057660.

[5] P. Ansah, C. John, S. K. Tetarave and E. Kalaimannan, "Enhancing network intrusion detection using graph convolutional networks fused with traditional systems," International Journal of Data Science and Analytics, 21(1), 31, 2025. doi:10.1007/s41060-025-00922-9.

[6] R. C. Nwachukwu, "Rethinking graph-based approaches: An empirical study of feature engineering in network intrusion detection," JTIF, 14(1), 1–10, 2026. doi:10.21063/jtif.2026.V14.1.1-10.

[7] E. Caville, W. W. Lo, S. Layeghy and M. Portmann, "Anomal-E: A self-supervised network intrusion detection system based on graph neural networks," Knowledge-Based Systems, 2022. arXiv:2207.06819.